

1. Descripción del Problema: "API de Cálculo de Envíos"

Deben implementar un sistema que calcule el costo de envío de un paquete. El sistema recibirá un archivo JSON con los detalles del pedido y deberá devolver el costo final, aplicando diferentes estrategias de cálculo y descuentos.

Entrada (Input) JSON: El sistema recibirá un JSON en un endpoint de la API con el siguiente formato:

```
JSON
{
  "items": [
    { "id": 1, "weight_kg": 0.5, "category": "electronics" },
    { "id": 2, "weight_kg": 2.0, "category": "books" }
  ],
  "destination": "international",
  "coupon": "PRIME_USER"
}
```

Donde:

- **destination:** Indica el destino del envío. Puede ser:
 - **local**
 - **national**
 - **international**
- **coupon:** Indica el cupón de descuento a aplicar.
 - (campo vacío o nulo): Sin descuento.
 - **PRIME_USER:** Descuento del 15%.
 - **NEW_USER:** Descuento fijo de \$5.

Salida (Output) JSON: La API deberá devolver un JSON con este formato:

```
JSON
{
  "base_cost": 0.0,
  "discount_applied": 0.0,
  "final_cost": 0.0
}
```

2. Requisitos Técnicos y de Diseño

A. Patrón de Diseño (Práctica de Patrones) El núcleo del sistema *debe* ser modular. Para esto:

1. Deberán implementar el **Patrón Strategy** para calcular el `base_cost`.
 - Debe existir una interfaz `ShippingStrategy`.
 - Deben crear implementaciones concretas: `LocalShippingStrategy`, `NationalShippingStrategy`, `InternationalShippingStrategy`.
 - *Reglas de ejemplo (pueden inventar las suyas):*
 - **Local:** $\$5 + (\$1 * \text{peso total})$.
 - **National:** $\$10 + (\$2 * \text{peso total})$.
 - **International:** $\$25 + (\$5 * \text{peso total})$.
2. Deberán implementar un segundo **Patrón Strategy** para el cálculo de descuentos (`discount_applied`).
 - Interfaz: `DiscountStrategy`.
 - Implementaciones: `NoDiscountStrategy`, `PrimeUserStrategy` (15%), `NewUserStrategy` (\$5 fijos).

B. Implementación de API (Práctica de Redes)

1. El sistema debe ser una API web desarrollada con **FastAPI**.
 2. Debe exponer, como mínimo, el siguiente endpoint:
 - `POST /calculate-shipping`: Recibe el JSON de entrada y retorna el JSON de salida.
 3. Deben incluir un archivo `requirements.txt` con `fastapi` y `uvicorn`.
-

3. Metodología y Colaboración (Práctica de Git Colaborativo)

Se evaluará el correcto seguimiento del flujo de trabajo colaborativo.

1. **Setup:** Un miembro del equipo crea un repositorio *privado* en GitHub y agrega al otro como colaborador.
2. **Issues:** Deben crear **Issues** en GitHub para todas las tareas a realizar (Ej: "Implementar Strategies de Envío", "Crear endpoint POST /calculate-shipping", "Configurar CI").
3. **Branches:** *Todo* el desarrollo debe hacerse en *feature branches* (ej: `feature/shipping-strategies`). La rama `main` debe permanecer protegida y funcional.
4. **Pull Requests (PRs):** Al finalizar una *feature*, deben abrir un **Pull Request** contra `main`.
5. **Revisión:** El otro miembro del equipo *debe* revisar el código, hacer comentarios (si es necesario) y **Aprobar** el PR.

6. **Merge:** Solo se puede hacer *merge* a `main` después de la aprobación y si la CI (ver punto 4) pasa con éxito.
-

4. CI/CD y Despliegue (Prácticas de CI/CD y Redes)

A. Integración Continua (CI)

1. Deben escribir **Unit Tests** (usando `unittest` o `pytest`) que prueben la lógica de las *Strategies* por separado.
2. Deben crear un *workflow* de GitHub Actions en `.github/workflows/ci.yml`.
3. Este *workflow* debe dispararse en cada `push` y `pull_request` a la rama `main`.
4. El *workflow* debe instalar las dependencias (`pip install -r requirements.txt`) y **ejecutar los tests**.

B. Despliegue Continuo (CD)

1. La API debe estar desplegada y funcional en **Render.com**.
2. Deben configurar el "Web Service" en Render para que se conecte a su repositorio de GitHub.
3. Configurar los comandos de Render:
 - **Build Command:** `pip install -r requirements.txt`
 - **Start Command:** `uvicorn main:app --host 0.0.0.0 --port 10000`

C. Release (Opcional pero recomendado)

1. Una vez que todo esté funcionando, crear un `tag` de Git (ej: `v1.0.0`) y hacer `push` del tag.
 2. Crear un segundo *workflow* de GitHub Actions (`cd_release.yml`) que cree automáticamente un "Release" en GitHub cuando se pushea un tag.
-

Entregables

Para aprobar, deberán presentar:

1. La URL de su repositorio de GitHub.
2. La URL de su API funcionando en Render.com (ej: `https://mi-examen.onrender.com`).